

x86 Architecture Overview

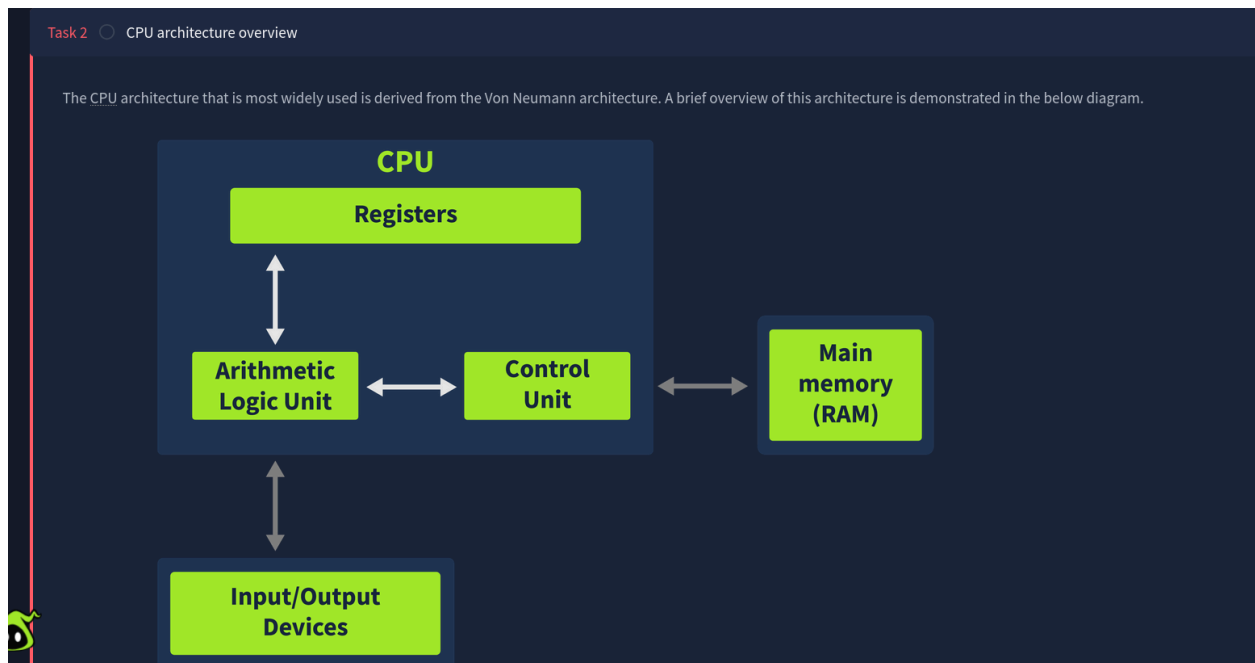
PREPARED by Sirajudin
CTC-153-26



x86 Architecture Overview

A crash course in x86 architecture to enable us in malware reverse engineering.

60 min 34,107



Control Unit:

The Control Unit gets instructions from the main memory, depicted here outside the

.

The address to the next instruction to execute is stored in a register called the Instruction Pointer or IP. In 32-bit systems, this register is called EIP, and in 64-bit systems, it is called RIP.

Arithmetic Logic Unit:

The arithmetic logic unit executes the instruction fetched from the Memory. The results of the executed instruction are then stored in either the Registers or the Memory.

Registers:

The Registers are the

's storage. Registers are generally much smaller than the Main Memory, which is outside the , and help save time in executing instructions by placing important data in direct access to the

.

Memory:

The Memory, also called Main Memory or Random Access Memory (

),

contains all the code and data for a program to run. When a user executes a program, its code and data are loaded into the Memory, from where the

accesses it one instruction at a time.

I/O devices:

I/O devices or Input/Output devices are all other devices that interact with a computer. These devices include Keyboards, Mice, Displays, Printers, Mass storage devices like Hard Disks and USBs, etc.

Answer the questions below

When a program is loaded into Memory, does it have a full view of the system memory? Y or N?

N

✓ Correct Answer

Which section of the Memory contains the code?

Code

✓ Correct Answer

Which Memory section contains information related to the program's control flow?

Stack

✓ Correct Answer

The Stack Pointer:

The Stack Pointer points to the top of the stack. When any new element is pushed on the stack, the location of the Stack Pointer changes to consider the new element that was pushed on the stack. Similarly, when an element is popped off the stack, the stack pointer adjusts itself to reflect that change.

The Base Pointer:

The Base Pointer for any program remains constant. This is the reference address where the current program stack tracks its local variables and arguments

High

Stack frame for caller

Argument 1

Argument 2

Return Address

Saved EBP

Local Var 1

Local Var 2

Local Var 3

Stack frame for callee

Argument 1

Argument 2

Return Address

Saved EBP

Local Var 1

Local Var 2

Local Var 3

EBP

ESP

Low



Old Base Pointer and Return Address:

Below the Base Pointer lies the old Base Pointer of the calling program (the program that calls the current program). And below the old Base Pointer lies the Return Address, where the Instruction Pointer will return once the current program's execution ends. A common technique to hijack control flow is to overflow a local variable on the stack such that it overwrites the Return Address with an address of the malware author's choice. This technique is called a Stack Buffer Overflow.

Arguments:

The Arguments being passed to a function are pushed to the stack before the function starts execution. These arguments are present right below the Return Address on the stack.

Function Prologue and Epilogue:

When a function is called, the stack is prepared for the function to execute. This means that the arguments are pushed to the stack before the start of the function execution. After that, the Return Address and the Old Base Pointer are pushed onto the stack. Once these elements are pushed, the Base Pointer address is changed to the top of the stack (which will be the Stack Pointer of the caller function at that time). As the function executes, the Stack Pointer moves as per the requirements of the function. This portion of code that pushes the arguments, the Return Address, and the Base Pointer onto the Stack and rearranges the Stack and Base Pointers is called the Function Prologue.

Answer the questions below

Follow the instructions in the attached static site and find the flag. What is the flag?

THM{SMASHED_THE_STACK}

✓ Correct Answer